

CSC3044S
Artificial Intelligence: AI Systems
Introduction I

Geoff Nitschke
gnitschke@cs.uct.ac.za

CSC3044S (AI Systems) Contacts

Lecturer Emails:

Geoff Nitschke (Section 1): gnitschke@cs.uct.ac.za

Francois Meyer (Section 2): francois.meyer@uct.ac.za

Melissa Densmore (Section 3): mdensmore@cs.uct.ac.za

TAs Email:

Oliver Foxcroft: FXCOLI001@myuct.ac.za

Max Van Veen: VVNMAX002@myuct.ac.za

AI Systems: Course Evaluation

- **Project: (25%): Released:** 14 September – **Due:** 12 October.
- **Test: (25%):** 2 October, 18.00–18.45; Venue: New LT.
- **Exam: (50%): 2 hours** – 12 November; Venue: Snape TS2B.
- **DP Requirement:** 45% aggregate in practical work.

AI Systems: Course Structure

- **Weeks 1-2:** AI Agents and Systems
- **Week 3:** Human-centered AI Systems & Societal Impacts I – Melissa Densmore
- **Week 4:** AI Systems Design – Francois Meyer
- **Week 5:** Project Presentations and Demos
- **Week 6:** Human-centered AI Systems & Societal Impacts II – Melissa Densmore

AI Systems: Course Evaluation – NOTE

- See course notes for all relevant course information!
- All requests for assignment extensions – Go to the course convenor: Me! (gnitschke@cs.uct.ac.za).
- All assignment and test mark queries – Go to the TAs.

AI Systems: Course Reading

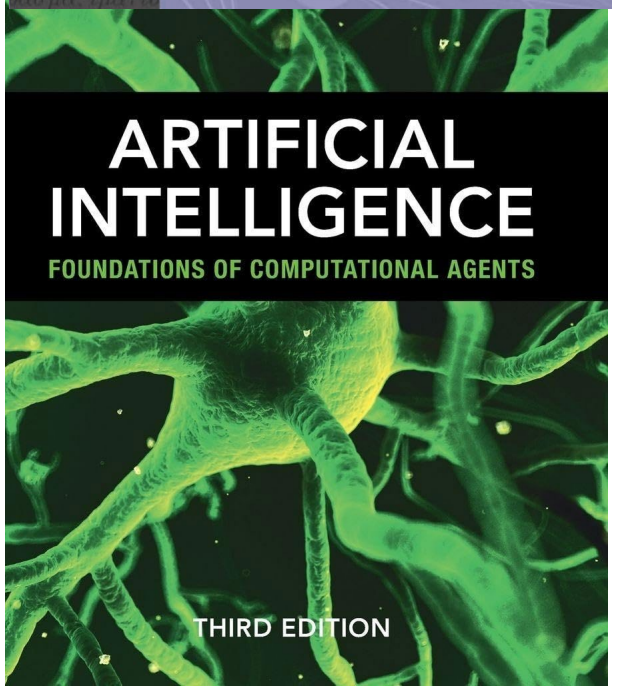
Recommended Books:

- Russell, S., and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* – 4th Edition: aima.cs.berkeley.edu/

- Poole, D., and Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents* – 3rd Edition: artint.info/3e/html/ArtInt3e.html

Reading: Chapters and papers assigned per lecture – On Amathuba!

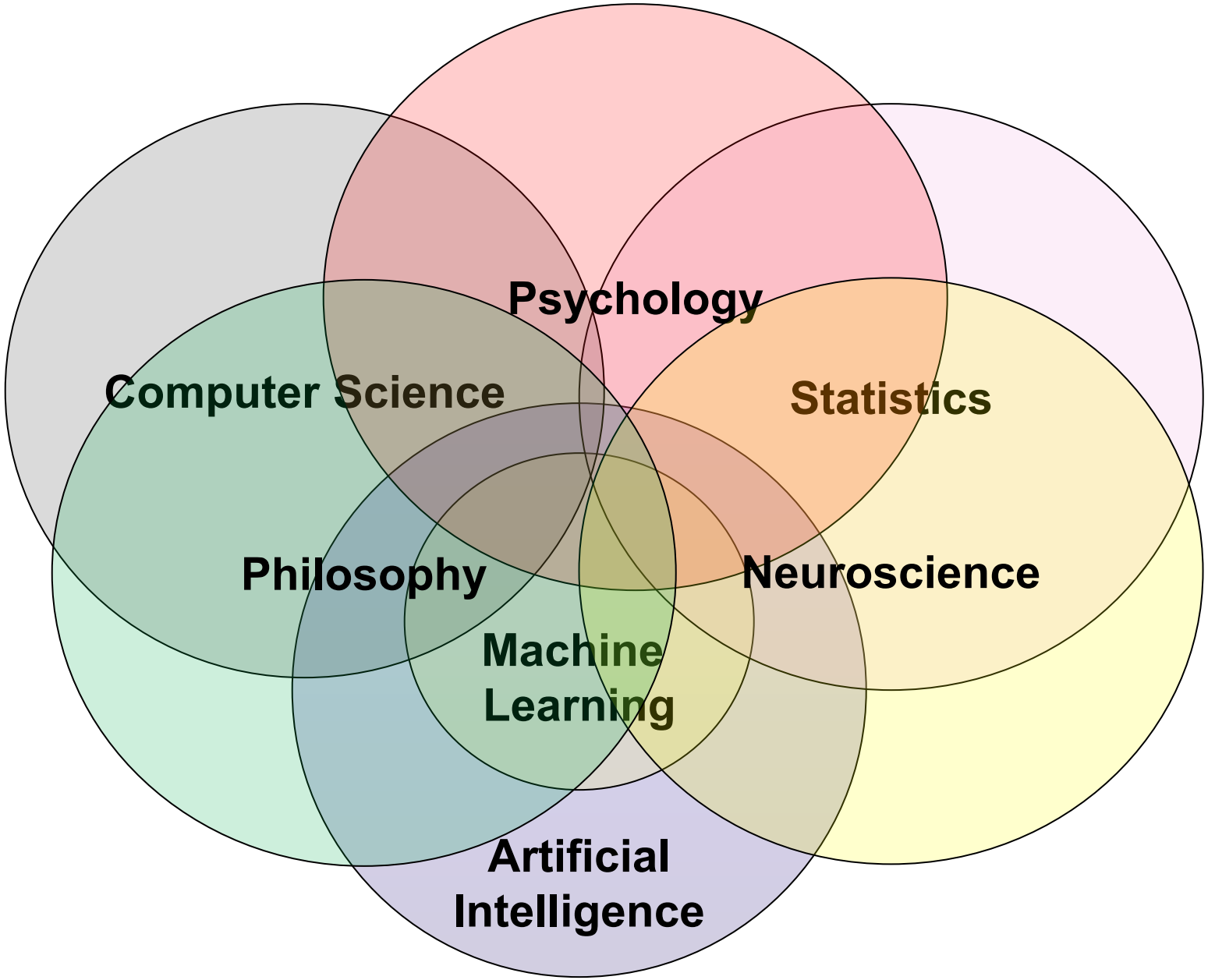




AI Systems: Course Syllabus

- **Section 1: Agents and Agent Architectures**
- Section 2: AI Systems Design
- Section 3: Human-centered AI Systems & Societal Impacts of AI

What is Artificial Intelligence (AI)?



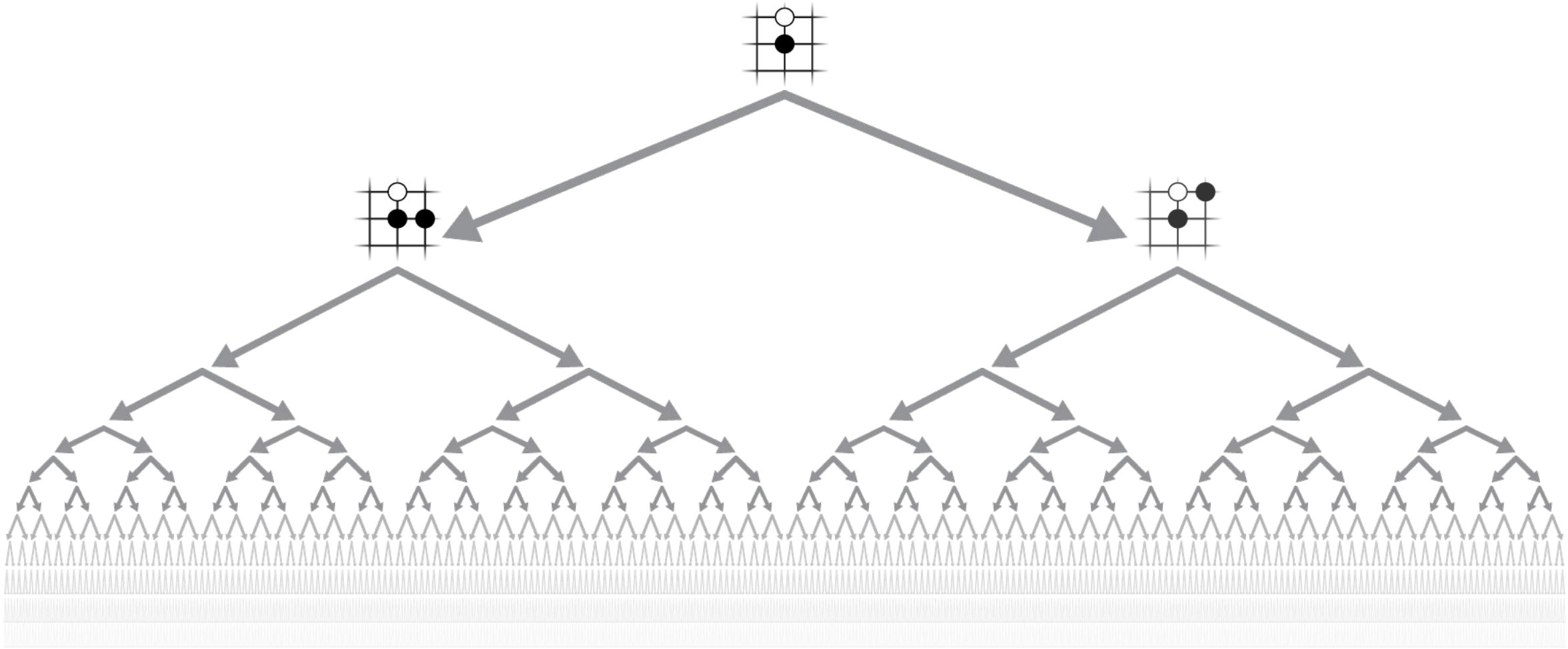
What is AI?

- Has its roots in many different disciplines – e.g.:
  - ***Bayes rule*** from probability.
  - ***Game theory*** from mathematics and economics.
  - ***Stochastic gradient descent*** from statistics.
  - ***Value iteration*** from control theory.
  - ***Temporal difference learning*** from psychology.
  - ***Artificial neural networks*** from neuroscience.
- A very successful approach (dominating AI research) is now…

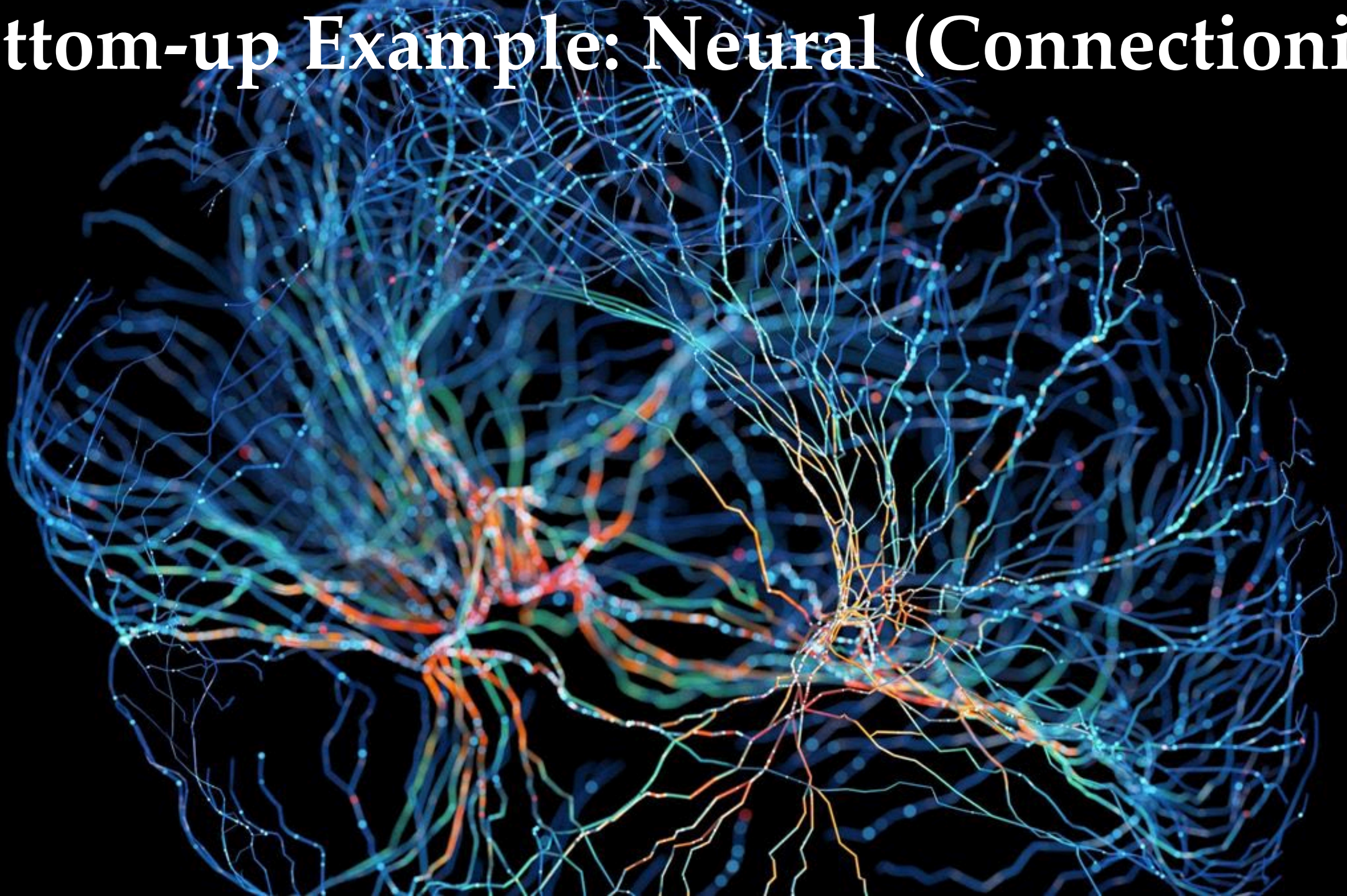
Neural AI: Beyond Symbolic AI

- **Symbolic AI:** Top-down approach (logic, rule-based).
- **Neural AI:** Bottom-up approach (connectionist, neuro-inspired).

Top-Down Example: Symbolic



Bottom-up Example: Neural (Connectionist)



The AI Renaissance

Symbolic AI + Neural AI + tools and ideas from other fields…

Optimization,
algorithms,
probabilistic inference,
statistics,
game theory, etc.

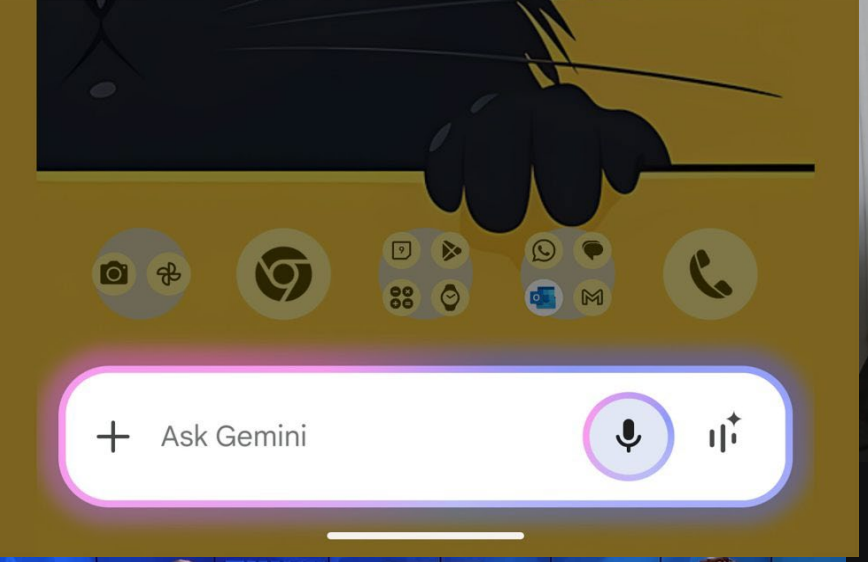
The AI Renaissance

Symbolic AI + Neural AI + tools and ideas from other fields…

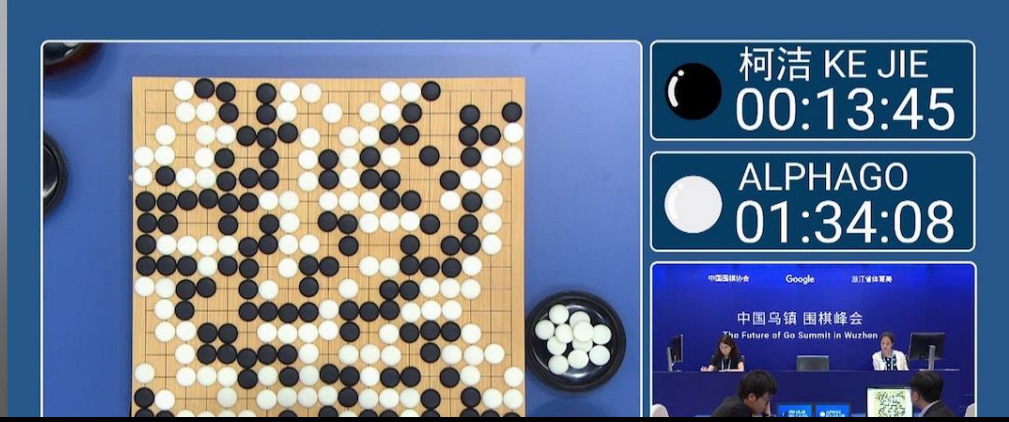
Agents ⟸

Optimization,
algorithms,
probabilistic inference,
statistics,
game theory, etc.

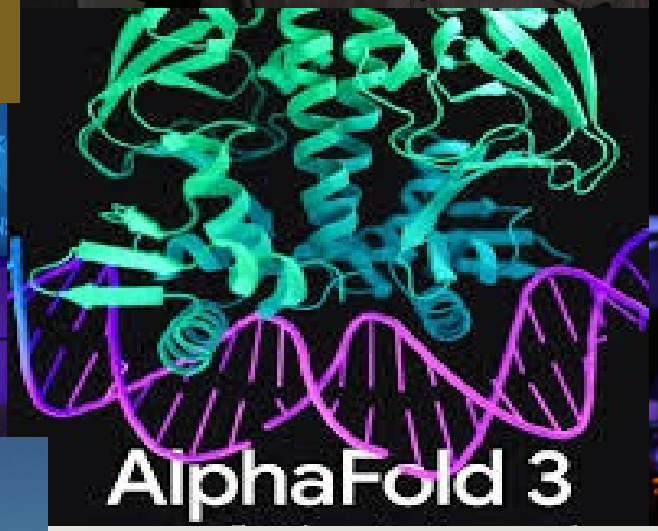
What is an Agent?


Ask Gemini




柯洁 KE JIE
00:13:45
ALPHAGO
01:34:08
中国乌镇 围棋峰会
The Future of Go Summit in Wuzhen


THINK
$24,000
Who is Stoker?
$1,000
$77,147
Who is Bram Stoker?
$ 17,973
$21,600
$5600


AlphaFold 3




MDV
MDV





A Brief History of Agents

- 1950s–present: ***Symbolic Reasoning Agents*****:** Decides what to do via symbol manipulation. Its purest expression, proposes that agents use explicit logical reasoning in order to decide what to do.
- 1980s–present: ***Reactive Agents*****:** Problems with symbolic reasoning led to the reactive agents.
- 1990s–present: ***Hybrid Agents*****:** Hybrid architectures attempt to combine the best of reasoning and reactive architectures.
- 2000s–present: ***Neuro (Cognitive) Agents***: … ?

Agents – Classic Definitions

Think like a human

- Cognitive science / neuroscience
- Does intelligence require biology?

Think rationally

- Logic and automated reasoning
- But, not all problems can be solved just by these techniques

Act like a human

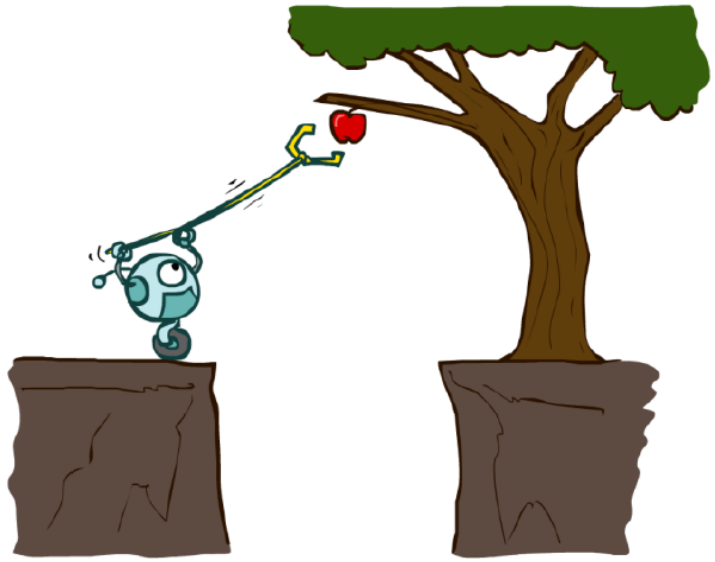
- Turing test
- ELISA, Loebner prize
- "What is 1228 x 5873"? … "I don't know, I'm just a human

Act rationally

- Basis for intelligent agents framework
- Unclear if this captures the current scope of AI research

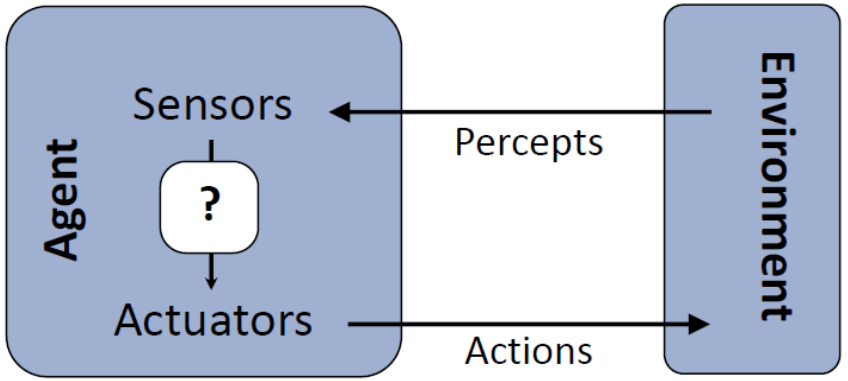
Designing Agents

- **Agent** is an entity that *perceives* and *act*



- Characteristics of the **percepts, state, environment,** and **action space** dictate methods for selecting actions

- What is the **"?"** between sensors and actuators?

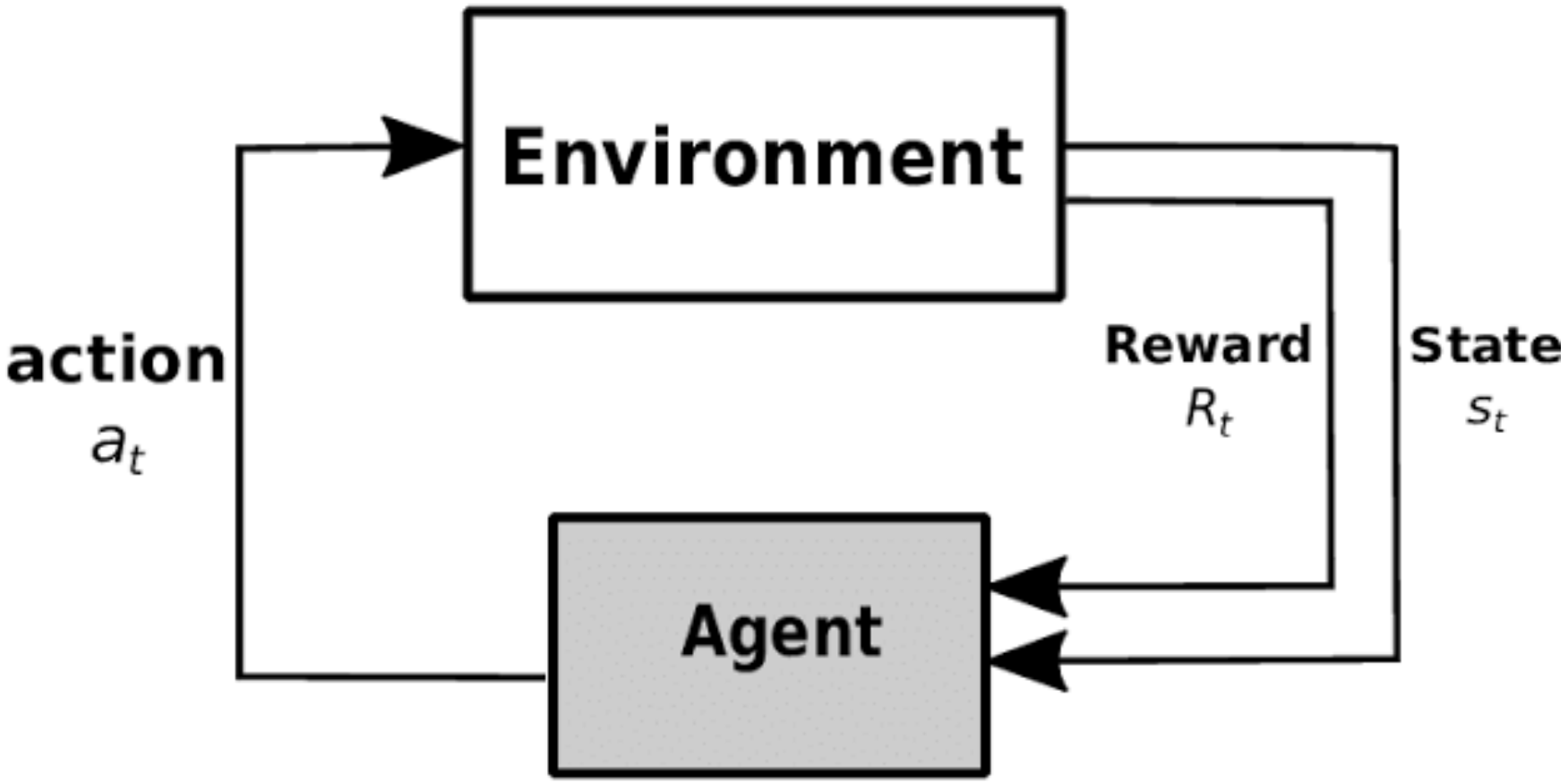


Rational Agents

- **Rational:** Maximally achieving pre-defined goals (objectives).
- *Rationality* only concerns *what decisions* are made (not the "thought" processes behind them).
- Goals are expressed in terms of the **utility** of outcomes.
- Being rational means **maximizing your expected utility.**

What is a Classic Rational Agent Algorithm?

Reinforcement Learning (Agent)

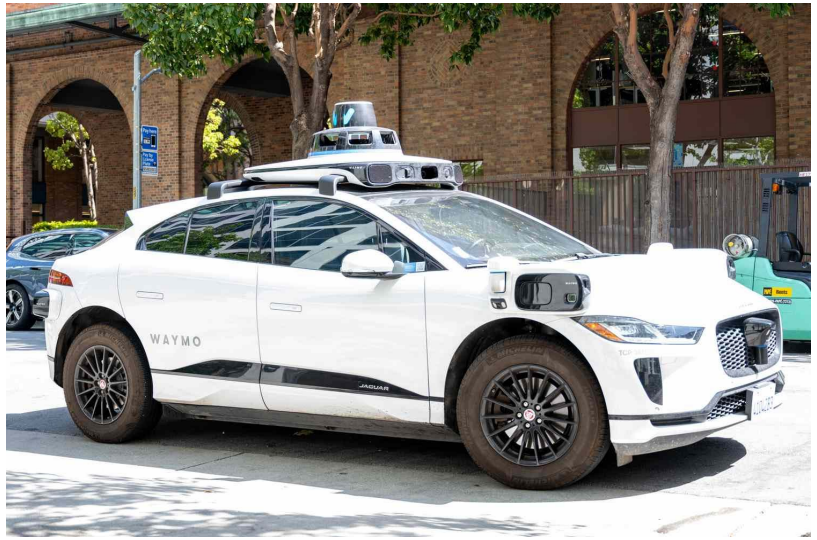


What are some Real-world examples of Rational Agents?

Examples of Rational Agents





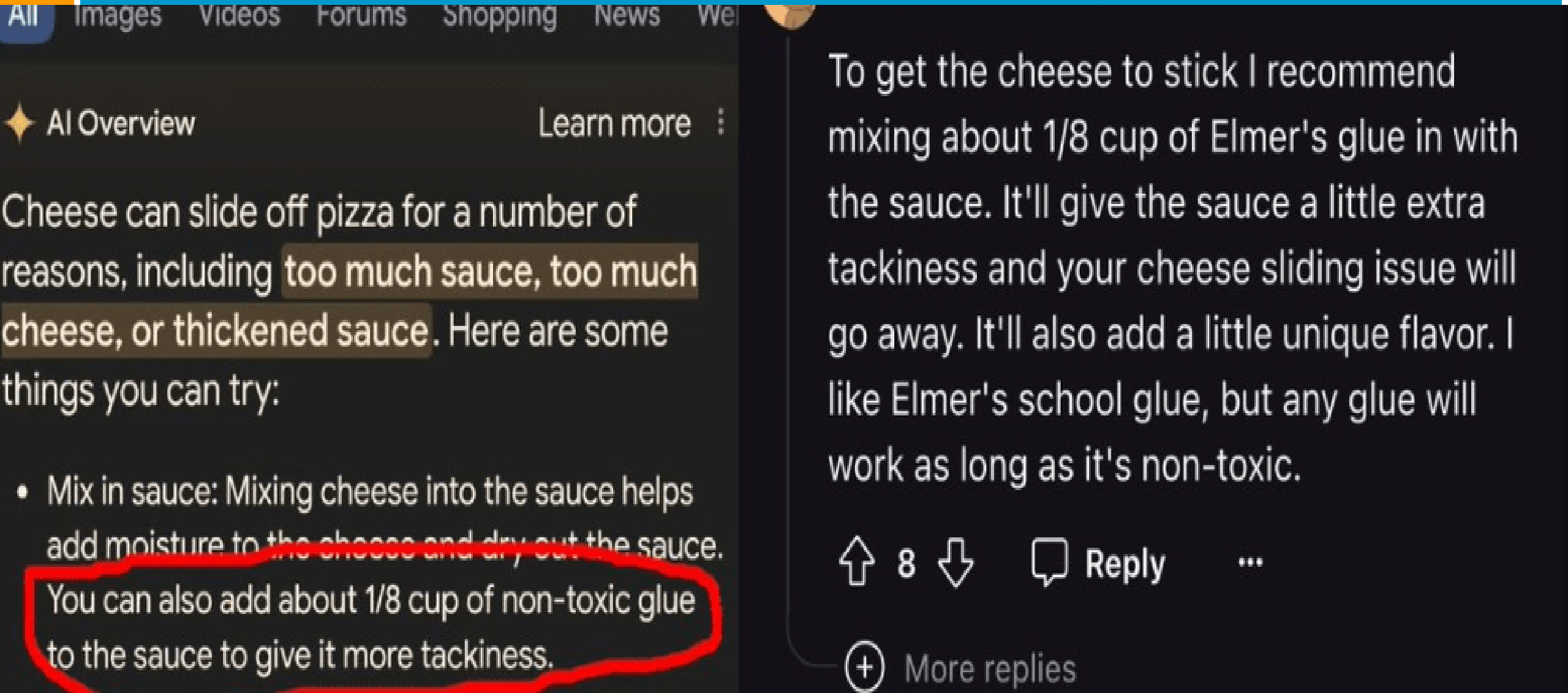






Irrational Agents?

Irrational Agents



Irrational Agents?

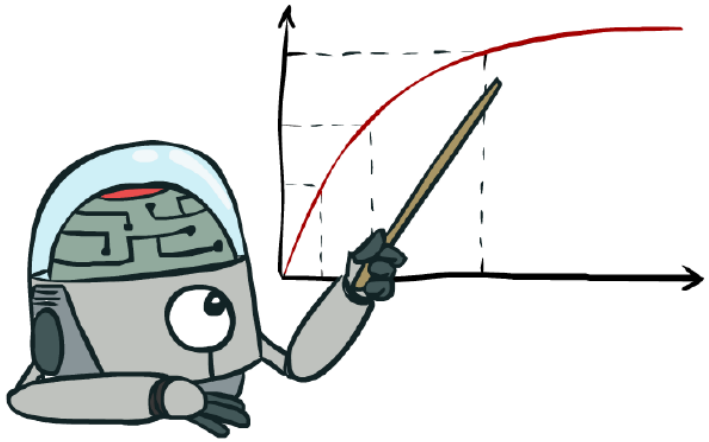
- *An agent takes actions leading to low utility (reward).*

- **Incomplete Information:** Agent's model of the world, (sensory) input, or training data is *too simple*, *not enough compute time*, *too much noise in training data or in inputs* – leading to poor outputs.

- **Hallucinations and Noise:** Given problems with training data or noise in the data – e.g., *Large Language Models* (LLMs) can produce inconsistent and fabricated outputs – ***irrational outputs!***

Back to Rational Agents

- What is rational depends on:
  - Performance metric (Objective function).
  - Agent's knowledge of environment (data).
  - Actions available to agent.
  - Sensory information.



- *Being rational means* ***maximizing your expected utility.***

Types of (Rational) Agents

- **Reactive** (Reflexive) agents – *Simple, Model-based*.
- **Deliberative** (Goal-based) agents.
  - **Learning** (RL, ANN, LLM,…) agents.

The 2026 (Rational) Agent Landscape

Family	What it does	Representative tools (September 2026)
Coding agents	Read, write, run, and debug code across a whole project, often unsupervised for long stretches.	Claude Code, OpenAI Codex, Cursor, GitHub Copilot, Cline, Devin
Computer-use agents	Drive a graphical interface — click, type, scroll — the way a person would, for tasks with no API.	Anthropic Computer Use, OpenAI Operator, Gemini Computer Use / Project Mariner
Research agents	Plan a multi-step search, gather and cross-check sources, and synthesise a cited report. “Deep Research” modes.	Deep Research modes in ChatGPT, Gemini, Claude and Perplexity; Grok DeepSearch
General agents	Attempt arbitrary multi-step tasks across tools and the web from a single instruction.	Manus (now Meta), ChatGPT Agent, Genspark, OpenManus
Creative agents	Drive creative software (3D, audio, design) through tool connectors.	Claude’s creative connectors (Adobe, Blender, Ableton, Autodesk Fusion)

Family	What it does	Representative tools (September 2026)
Coding agents	Read, write, run, and debug code across a whole project, often unsupervised for long stretches.	Claude Code, OpenAI Codex, Cursor, GitHub Copilot, Cline, Devin
Computer-use agents	Drive a graphical interface — click, type, scroll — the way a person would, for tasks with no API.	Anthropic Computer Use, OpenAI Operator, Gemini Computer Use / Project Mariner
Research agents	Plan a multi-step search, gather and cross-check sources, and synthesise a cited report. “Deep Research” modes.	Deep Research modes in ChatGPT, Gemini, Claude and Perplexity; Grok DeepSearch
General agents	Attempt arbitrary multi-step tasks across tools and the web from a single instruction.	Manus (now Meta), ChatGPT Agent, Genspark, OpenManus
Creative agents	Drive creative software (3D, audio, design) through tool connectors.	Claude’s creative connectors (Adobe, Blender, Ableton, Autodesk Fusion)

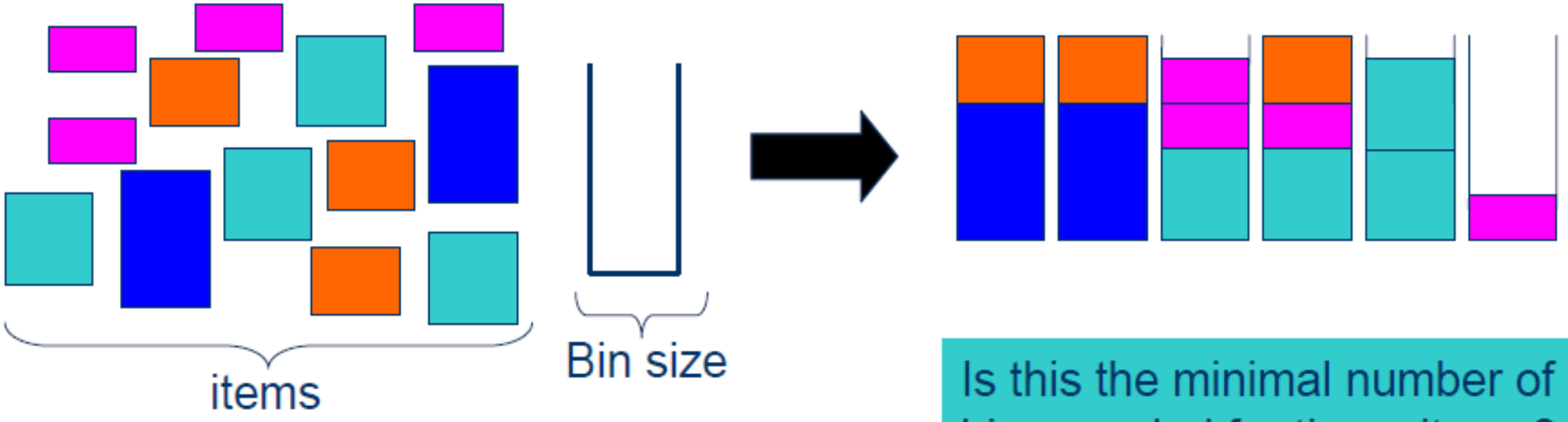
So, What else can Agents do?

Solving Intractable Problems

- Solve logistics (e.g., factory packing) problems.
- Play games (e.g., Sudoku, Go…)
- Vehicle routing (e.g., Multi-modal Traveling Salesman Problem).
- Design engineering products (e.g., design a satellite antenna).
- Robotic solutions (e.g., drive autonomous vehicles).
- …

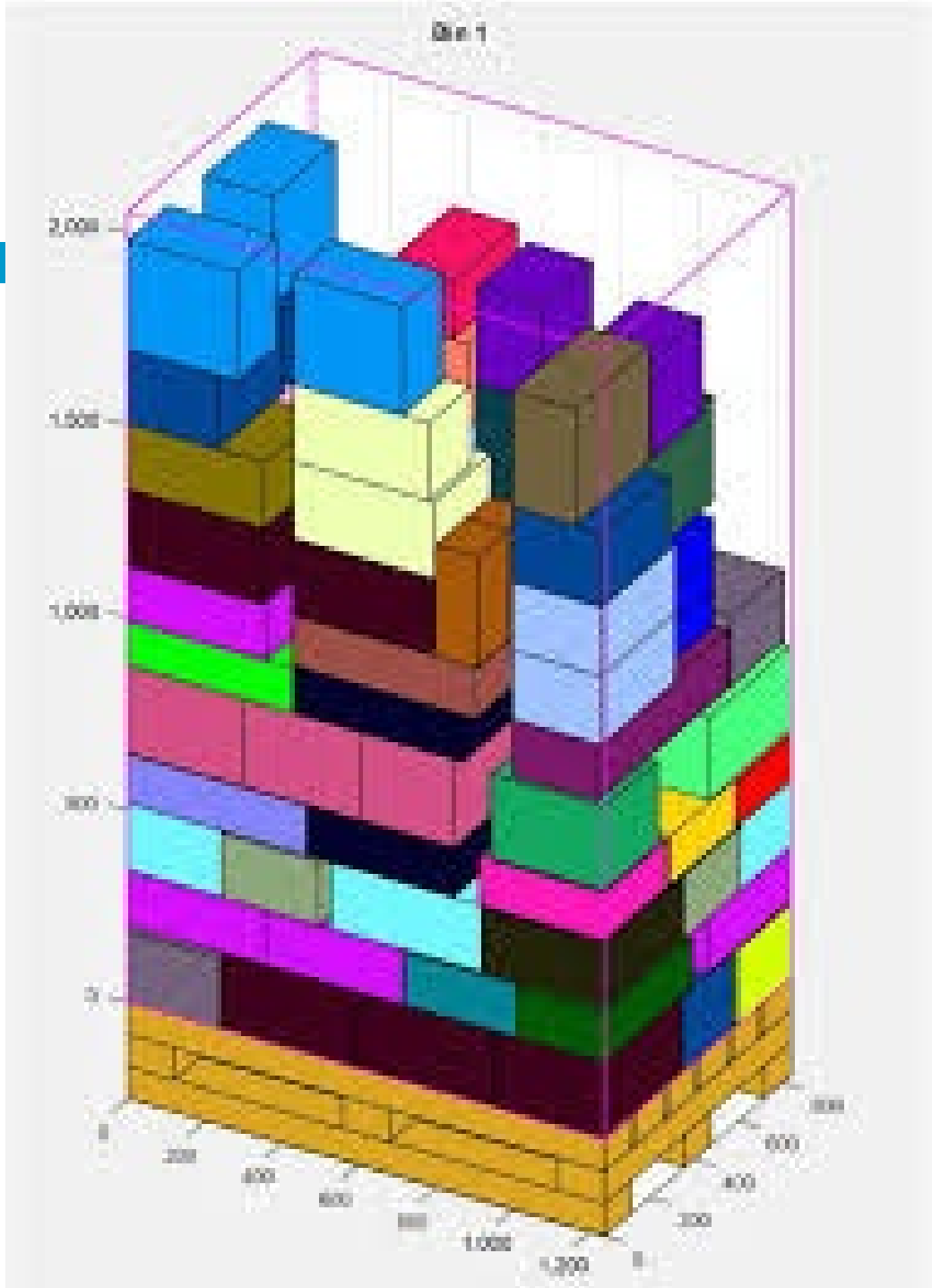
Logistics Problems

Given n items of different (1D) sizes, and given some fixed-capacity bins, pack the items into a **minimum** number of bins

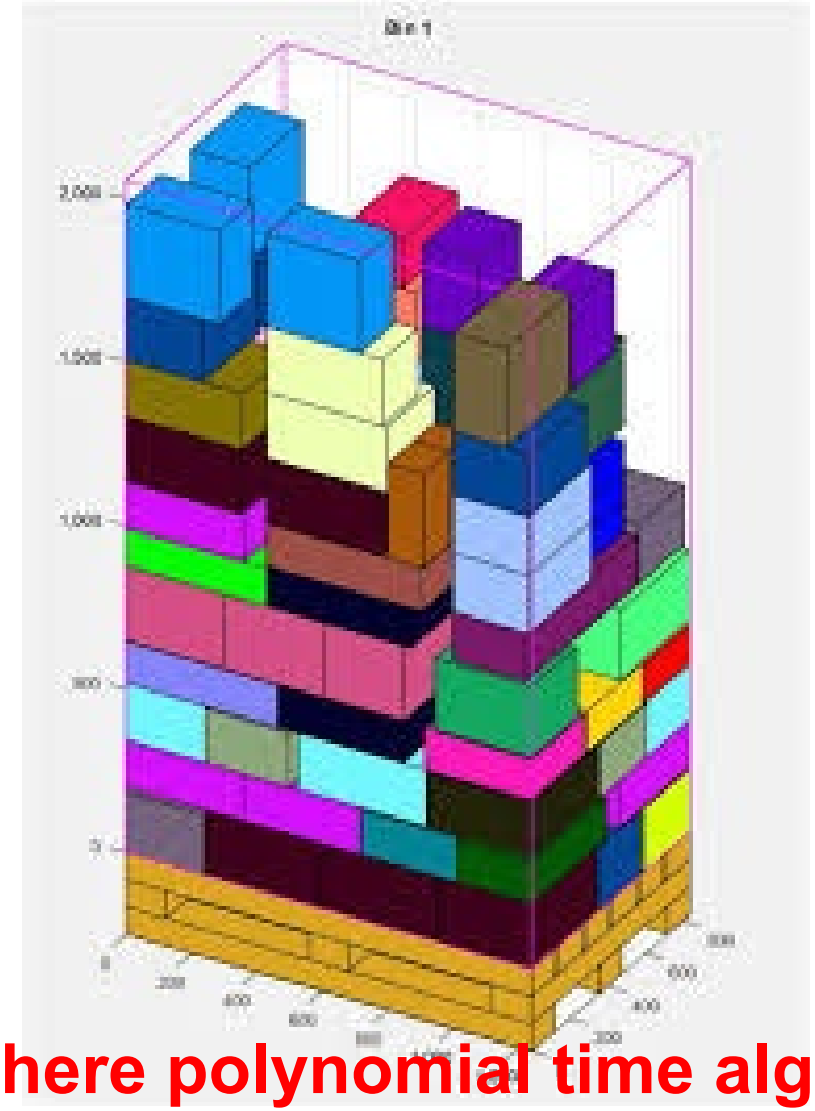


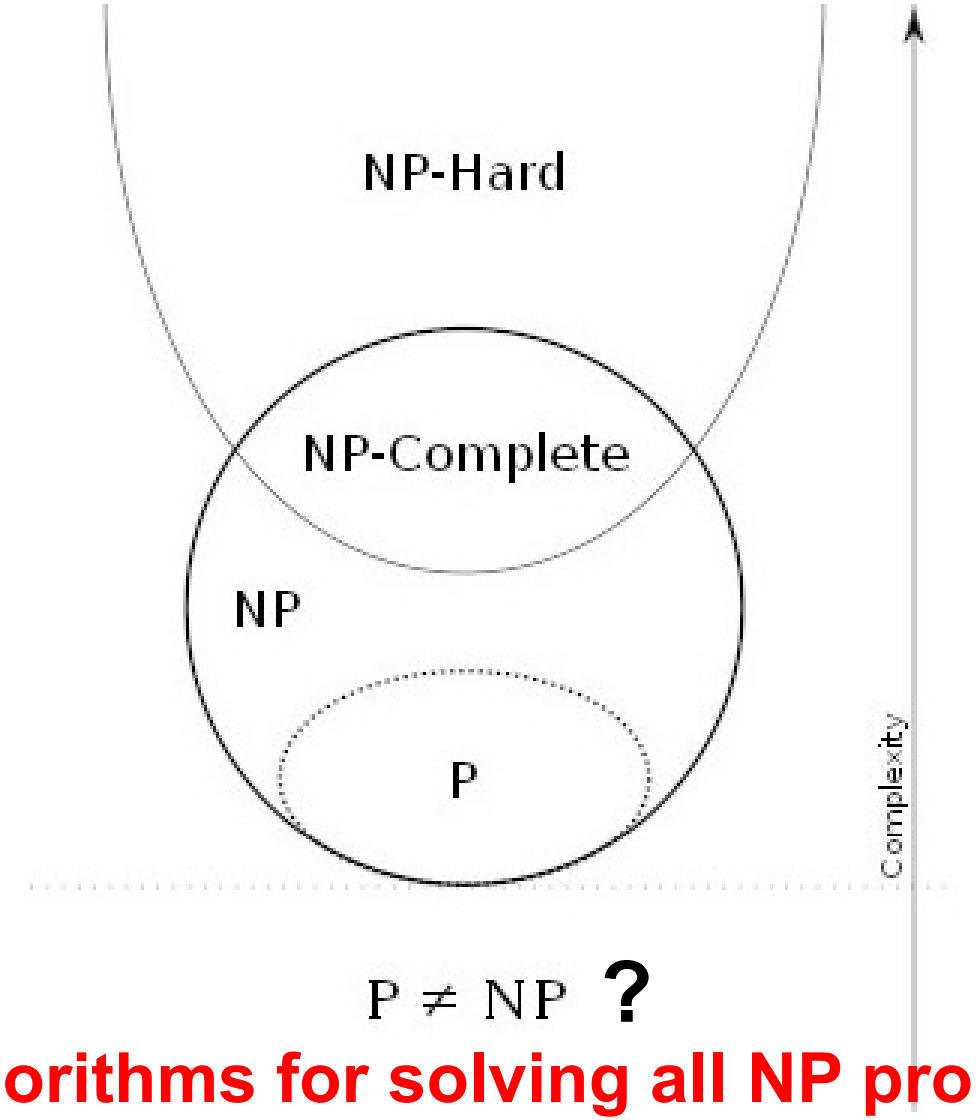
Is this the minimal number of bins needed for these items?

What type of (Computational complexity) problem is this?



NP-Problems





Are there polynomial time algorithms for solving all NP problems?

What makes a Problem P versus NP?

- **P (Polynomial):** Problems solved in *Polynomial time* (run time increases proportional to a power of the input size – e.g., n^2).

- **NP (Non-deterministic Polynomial):** Solutions can be *verified* quickly, even if finding solutions takes a long time (found in a non-deterministic way).

  - Example…?

What makes a Problem NP?

- **NP (Non-deterministic Polynomial):** Solutions can be *verified* quickly, even if finding solutions takes a long time.

- **Traveling Salesman Problem:** Shortest path between *n* cities.

  - **Verifiable:** One can quickly compute the path between *n* cities, to verify it is the shortest found thus far.

  - **Difficulty:** Time to find the shortest path between *n* cities, increases exponentially with *n*.

What makes a Problem NP Complete?

- **NP-Complete:** Solution Validity can be verified quickly (*in polynomial time*) – Brute-force search can find a valid solution by trying all possible solutions.

- What does the **"complete"** mean here?

What makes a Problem NP Complete?

- **“Complete”:** Problem is as universal and tough as any other problem in the NP group – If you can find a fast, efficient way to solve *one* NP-complete problem – Can use that method to solve *all* other NP problems.
  - Example…?

What makes a Problem NP Complete?

- **NP-Complete:** Solution validity is verified quickly (*polynomial time*). Brute-force search finds a valid solution by trying all possible solutions.

- **Boolean Satisfiability Problem:** Assign (true, false) to *n* variables to make a given Boolean formula **true**.

  - **Verification:** Given a proposed solution (specific combinations of *true, false* values) – one can verify that the formula evaluates to **true**.

  - **Difficulty:** Time to find a solution increases exponentially with *n*.

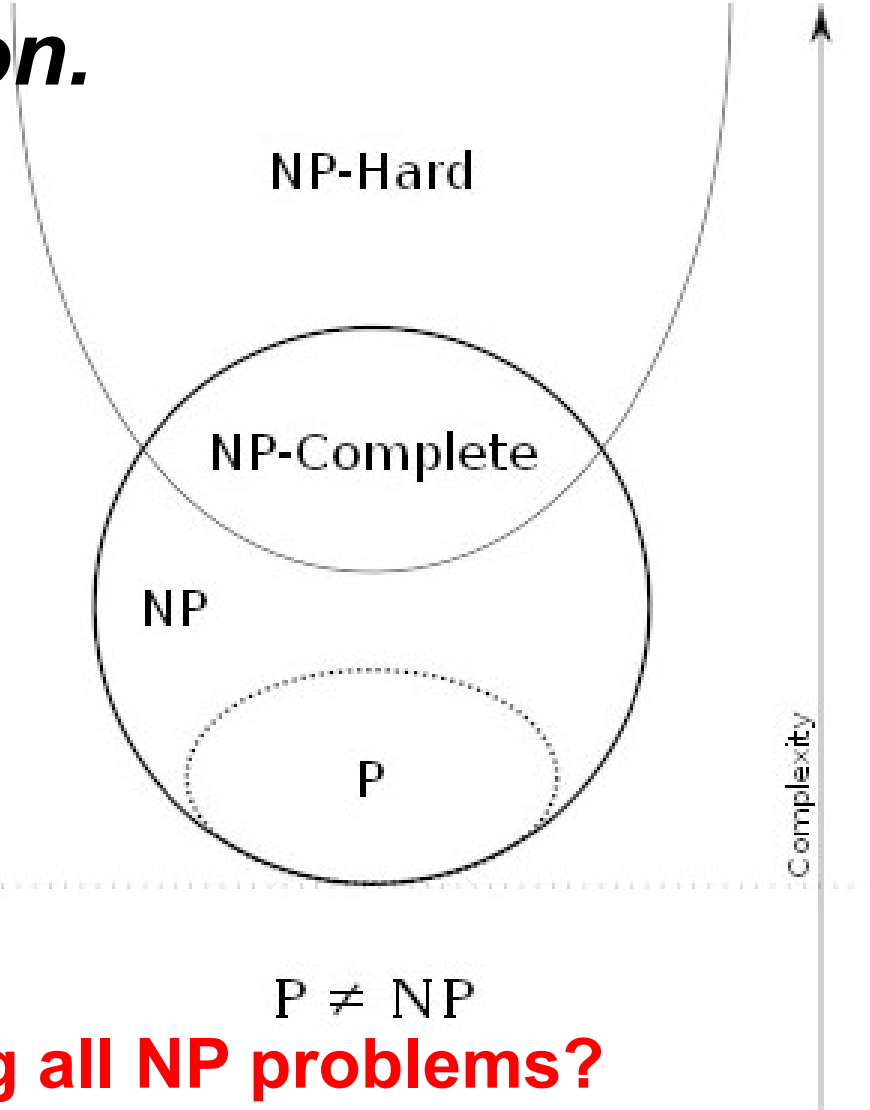
What makes a Problem NP-Hard?

- At least as hard as NP-complete problems – but may not be solved (and verified) in *Non-deterministic Polynomial* (NP) time.

- Example…?

What makes a Problem NP Hard?

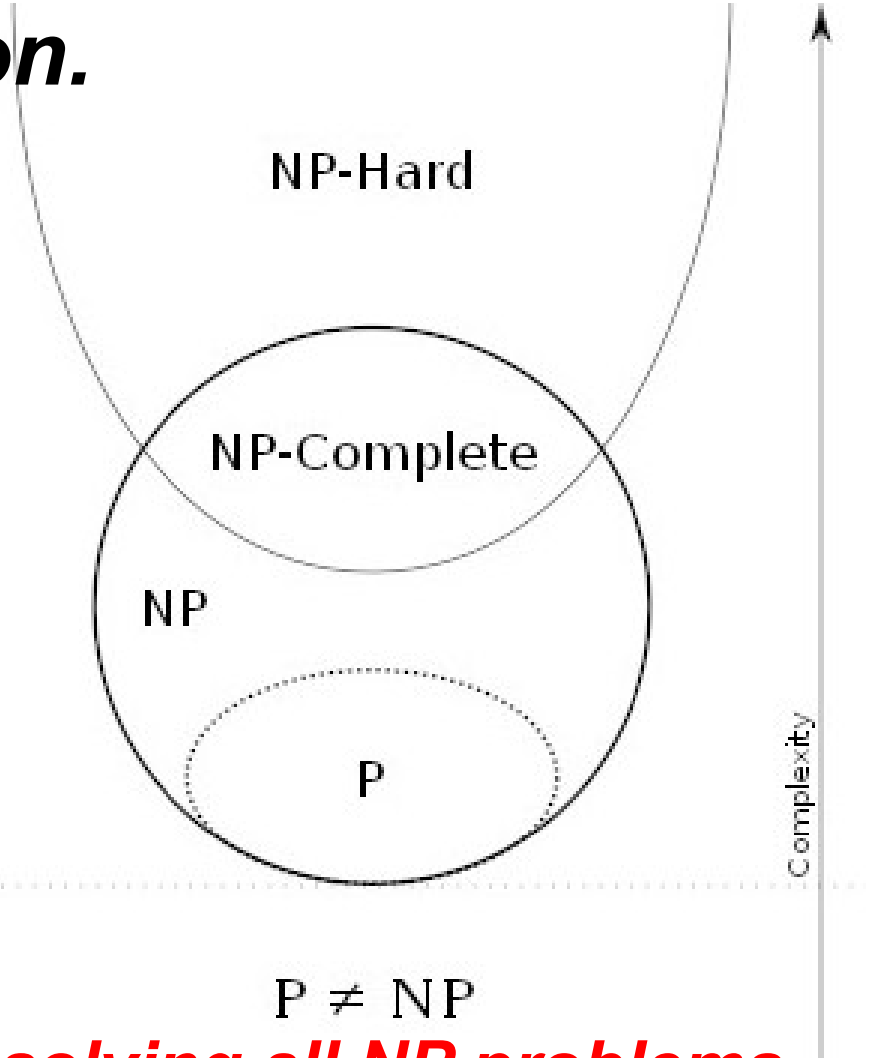
- **NP-hard:** ***No known efficient solution.***
  - **e.g.: Vehicle Routing Problem.**
  - Every NP-complete problem is NP-hard, but not all NP-hard problems are in NP.



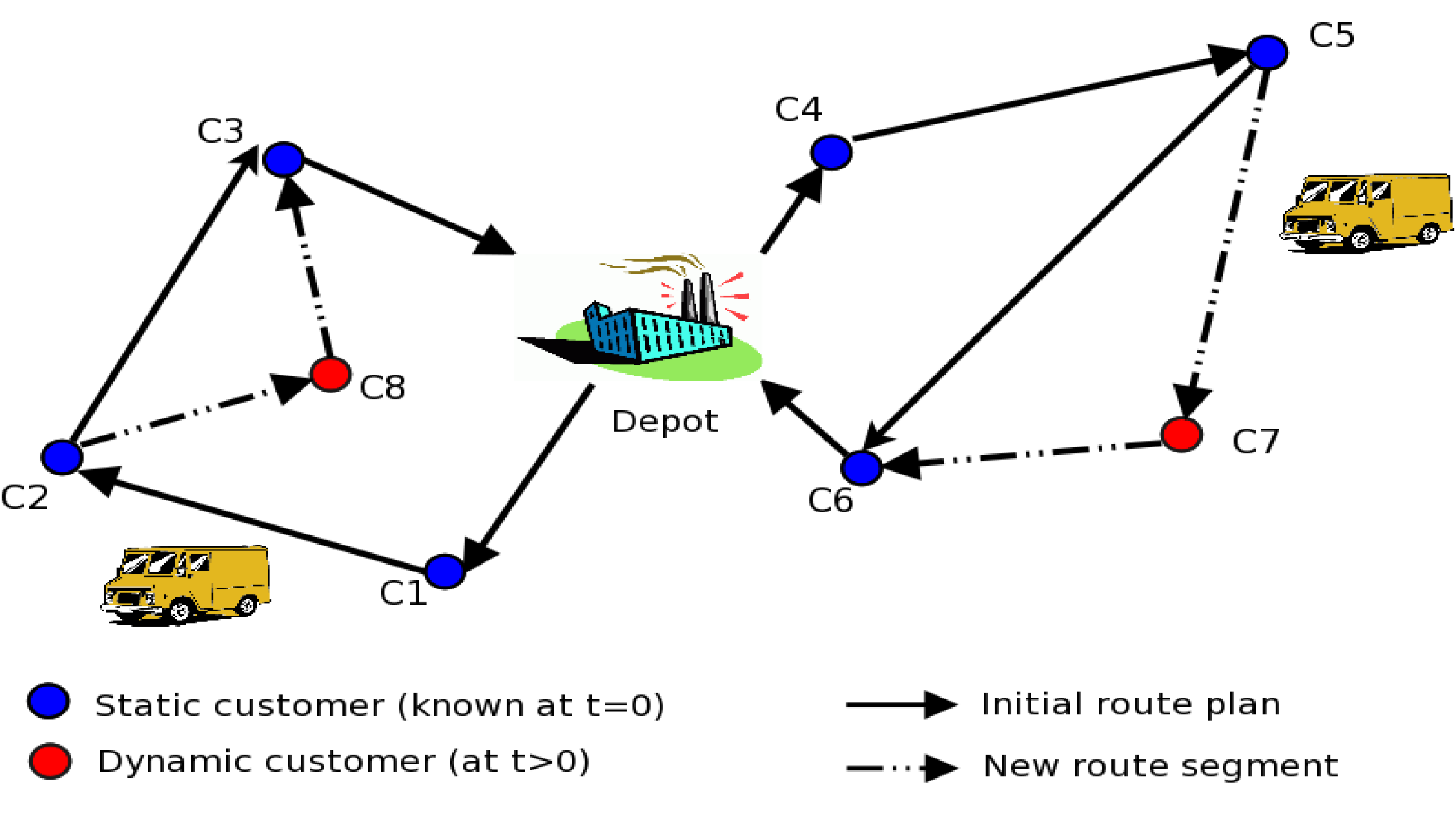
Are there polynomial time algorithms for solving all NP problems?

What makes a Problem NP Hard?

- **NP-hard:** ***No known efficient solution.***
  - **e.g.: Vehicle Routing Problem.**
  - Every NP-complete problem is NP-hard, but not all NP-hard problems are in NP.



There are no polynomial time algorithms (yet!) for solving all NP problems


C5
C4
C3
C8
Depot
C2
C7
C6
C1
Static customer (known at t=0)
Dynamic customer (at t>0)
Initial route plan
New route segment

Traveling Salesman Problem

- **NP-hard problem (no known efficient solution):**
  - ***n*** cities, visit each only once, minimise tour length.
- **Large search space:**
  - 10 cities, 10! = 3 628 800 possible tours.
  - 50 cities, 50! ≈ 10^{64} possible tours.
- **Time taken to solve a TSP (e.g. n=50)**

???

Traveling Salesman Problem

- **Time taken to solve a TSP (n=50):**
  - Simple enumeration: Proportional to n!
  - **e.g.:** If CPU executes 10^{12} operations per second.
  - **50 city tour ≈** $\frac{50!}{10^{12}} \approx 3 \times \frac{10^{64}}{10^{12}} = 3 \times 10^{52}$ secs **(≈ 31000 years)**

NP-Hard Optimisation

- Search for best solution $\mathbf{x}^*$ out of some fixed set **S**.
- Given computation, memory and time constraints:
  - *How do we find this best solution?*

AI Project
(14 September → 12 October)

Solving the Vehicle Routing Problem (VRP)

Project: Hand-ins & Milestones

- **Weeks 1–4:** Weekly code repo commits.
- **Week 5:** Final code hand-in.
- **Weeks 5–6:** Presentations.

Project: Evaluation (Agent Solution)

- Agent must design an optimiser module for solving the VRP.
- Plug into a VRP framework – Test on 10 (known) maps.
- **Final score:**
  - How close is your task performance (average over 10 unknown maps) to a **best** score?
  - e.g.: Same or higher = 100%; 0.1 of best score = 10%.

Project: Evaluation

- Weekly code repo commits: 5%
- Final score: 25%
- Presentation: 30%
- Session history (prompts): 40%

Summary: What you should know

- What is an Agent
- Types of Agents
- What agents can and cannot do.

Reading (On Amathuba!)

- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* – 4th Edition – Ch2 (Intelligent Agents): aima.cs.berkeley.edu/

- Poole, D., & Mackworth, A. (2023) *Artificial Intelligence: Foundations of Computational Agents* – 3rd Edition – Ch1 (Artificial Intelligence and Agents): https://artint.info/3e/html/ArtInt3e.Ch1.html

